

Hitec University Taxila, Department of Computer Engineering

Lab Manual for Mobile Application Development

Lab-9 ViewPager

Contents

1	Introduction	3
2	Activity Time boxing	3
3	Objectives of the experiment.....	3
4	Walk through Task	3
5	Home Task	7

1 Introduction

ViewPager is a class/layout manager that allows the user to swipe/flip left and right through pages of application.

Adapter

The adapter is used to generate the fragment or pages on the ViewPager

Tabs

To make the ViewPager more user friendly, the tabs can be added on top or bottom of the view pager

2 Activity Time boxing

Table 1: Activity Time Boxing

Task#	Activity Name	Activity time	Total Time
1	Introduction	60mins	60min
2	Walk Through Task	60mins	60min
3	Evaluation Task	60min	60min

3 Objectives of the experiment

- To get basic understanding regarding view pager
- To get basic understanding regarding how fragments used in view pager
- To get to know about the benefits related to view pager

4 Walk through Task

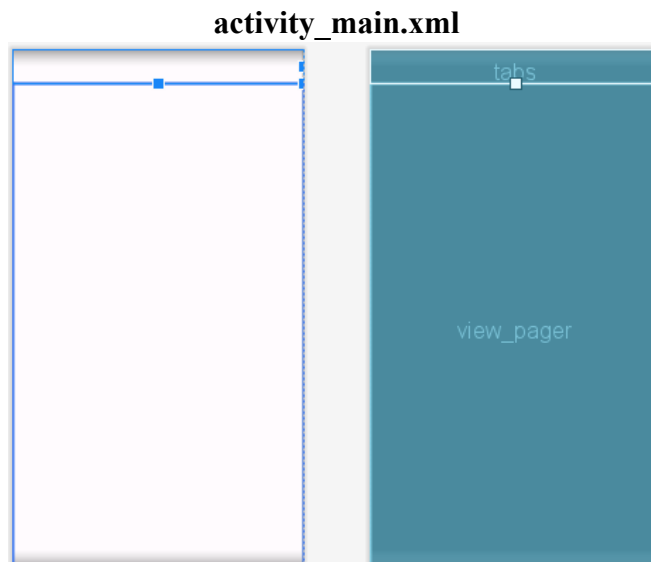
Creating New Activity

Step 1: Create Activities:

Create a new project just like you have created in your previous labs and create one more empty activity.

Step 2: Design XMLs of Activity:

Designs of activity are given below, just use the given xml code which is given in this step.



```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context="ViewPager.ViewPagerMain">

    <com.google.android.material.tabs.TabLayout
        android:id="@+id/tabs"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <androidx.viewpager.widget.ViewPager
        android:id="@+id/view_pager"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />
</LinearLayout>

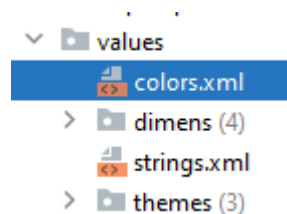
```

Step 3: Update colors.xml

```

<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
    <color name="green">#FF00FF00</color>
    <color name="blue">#FF0000FF</color>
</resources>

```



Step 3: Write XML for two fragments



```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="@dimen/view_margin"
    android:background="@color/green"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/fragment_title_a"
        android:gravity="center"
        android:textSize="32sp"
        android:textColor="@color/white"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:text="Fragment A" />

</LinearLayout>

```



```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="@dimen/view_margin"
    android:background="@color/blue"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/fragment_title_b"
        android:gravity="center"
        android:textSize="32sp"
        android:textColor="@color/white"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:text="Fragment B" />

</LinearLayout>

```

Step 4: Write fragments classes

FragmentA.java

```
public class FragmentA extends Fragment {
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable
        ViewGroup container, @Nullable Bundle savedInstanceState) {
        View fragmentView = inflater.inflate(R.layout.fragment_a_layout,
            container, false);
        return fragmentView;
    }
}
```

FragmentB.java

```
public class FragmentB extends Fragment {
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable
        ViewGroup container, @Nullable Bundle savedInstanceState) {
        View fragmentView = inflater.inflate(R.layout.fragment_b_layout,
            container, false);

        return fragmentView;
    }
}
```

Step 5: Write view pager adapter class

ViewPagerAdapter.java

```
public class ViewPagerAdapter extends FragmentPagerAdapter {

    public ViewPagerAdapter(FragmentManager fm) {
        super(fm);
    }
    @NonNull
    @Override
    public Fragment getItem(int position) {
        if(position == 0){
            return new FragmentA();
        } else if(position == 1){
            return new FragmentB();
        } else {
            return null;
        }
    }
    @Override
    public int getCount() {
        return 2;
    }

    @Nullable
    @Override
    public CharSequence getPageTitle(int position) {
        if(position == 0){
            return "FragmentA";
        }
    }
}
```

```

    } else if(position == 1){
        return "FragmentB";
    } else {
        return null;
    }
}
}

```

Step 6: Write main activity class

ViewPagerAdapterMain.java

```

public class ViewPagerMain extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_view_pager_main);

        ViewPagerAdapter adapter = new
ViewPagerAdapter(getSupportFragmentManager());

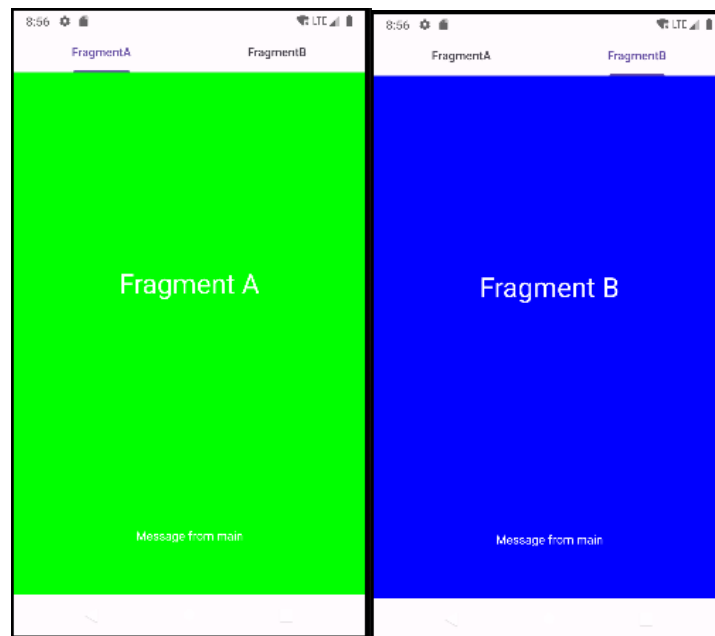
        ViewPager viewPager = findViewById(R.id.view_pager);
        TabLayout tabLayout = findViewById(R.id.tabs);

        tabLayout.setupWithViewPager(viewPager);
        viewPager.setAdapter(adapter);
    }
}

```

Step 6: Now Run Project

Click "Run" or press "Shift + F10".



5 Home Task

Use Tasbeeh app and show history in separate fragment in view pager.

Note: Bundle can share string array list.